

# Smart Video Management System (VMS)

**AetherShield** — Enterprise-grade AI surveillance platform for live monitoring, intelligent event detection, and timeline-linked investigation.

## 1. Problem Context

Traditional CCTV workflows separate live monitoring from recorded playback, forcing operators to switch tools during incidents and manually scrub hours of footage. False alerts from unbounded motion detection further degrade response quality. This case study required a unified prototype that combines live feeds, historical playback, AI intrusion/motion detection, configurable detection zones, timeline navigation, an event list, and an operations dashboard.

## 2. Solution Overview

**AetherShield VMS** is a full-stack prototype inspired by commercial platforms (Verkada, Eagle Eye Networks, Milestone). Operators monitor multi-camera live grids, investigate via a colored AI event timeline, draw detection zones with sensitivity controls, and jump from any event to the exact recording window (–10s / +20s). Beyond the brief, the system adds natural-language search, face & plate intelligence, incident workflow, heatmaps, PDF reports, and role-based access — while remaining runnable locally for demos.

VIDEO DEMONSTRATION LINK  
[ INSERT YOUR VIDEO URL HERE ]  
Paste the Unstop / YouTube / Drive link before final submission.

## 3. System Architecture

Cameras (sample files, laptop webcam, RTSP/HTTP remote devices) feed parallel pipelines: **live MJPEG streaming** for the operator UI, and an **AI detection worker** (YOLOv11 + ByteTrack) that writes structured events to SQLite. A timeline generator indexes events by camera and timestamp. The React dashboard consumes REST + WebSocket alerts for real-time operations. Recordings are segmented MP4 files with metadata for seek/replay.

| Layer       | Technology                                                           |
|-------------|----------------------------------------------------------------------|
| Frontend    | React 19, TypeScript, Vite, Tailwind CSS, Recharts, Zustand          |
| Backend     | FastAPI (Python), SQLAlchemy, JWT + RBAC (Admin / Operator / Viewer) |
| AI / Vision | Ultralytics YOLOv11n, ByteTrack, OpenCV zones, face match, plate OCR |
| Media       | OpenCV capture → MJPEG live + MP4 segment recording                  |
| Data        | SQLite (Postgres-ready), local storage for clips/snapshots           |
| Deploy      | Docker Compose ready; local Windows start scripts for demos          |

## 4. Data Flow (simplified)

Cameras → Live Stream + YOLO/Track → Zone Intrusion Check → Event DB / Alerts / Incidents → Timeline + Dashboard + Playback Seek → AI Search / Chat / PDF

### 5. Case Study Requirements — Coverage

Every mandatory capability is implemented and demonstrated in the product UI:

|                                                                                                                                 |                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Live camera feeds</b><br><br>Multi-camera MJPEG grid with 1 / 2 / 4 / 9 / 16 layouts; webcam, RTSP, and HTTP device sources. | <b>Recorded playback + timeline</b><br><br>Scrollable colored markers; click seeks recording with –10s replay context.  |
| <b>AI intrusion / motion</b><br><br>YOLOv11 object classes + motion fallback; zone-scoped intrusion = critical priority.        | <b>Detection zones</b><br><br>Draw rectangle/polygon/circle; sensitivity, enable/disable, class filters.                |
| <b>Event list</b><br><br>Timestamp, camera, confidence, track ID, priority; one-click evidence open.                            | <b>Operations dashboard</b><br><br>Camera health, today's detections, storage forecast, CPU/RAM, alerts, weekly charts. |

### 6. Differentiating Capabilities (Beyond the Brief)

To resemble a commercial VMS rather than a classroom prototype, AetherShield includes:

- **Natural-language AI search** — e.g. “Show all people after 9 PM”, “intrusions today”.
- **Chat with video** — RAG over event metadata; 24-hour AI summary.
- **Face intelligence** — enroll known / blacklist; unknown face alerts.
- **License plate search** — OCR when vehicles appear; plate → clip navigation.
- **Incident management** — open / investigating / resolved / false alarm / escalated.
- **Live WebSocket alerts** — in-app toasts + browser notifications on critical events.
- **Activity heatmap** — detection density overlay per camera.
- **Clip export & PDF security report** — evidence packaging for operators.
- **Demo vs live camera control** — toggle sample feeds OFF when using real cameras only.
- **RBAC** — Admin, Operator, Viewer permission sets with JWT authentication.

### 7. Implementation Approach

Development prioritized a reliable end-to-end demo path first (seeded cameras, synthetic videos, auth, dashboard, playback), then accuracy and operator UX (zones, tracking, event→seek), then portfolio differentiators (NL search, faces/plates, incidents, remote device connect). AI runs in per-camera worker threads with cooldowns to avoid event spam. When YOLO tracking dependencies are unavailable, the pipeline degrades gracefully to prediction-only or motion detection so demos never hard-fail.

**Camera operations:** Operators can connect this PC’s webcam, remote phones (HTTP MJPEG), or IP cameras (RTSP), disconnect live sources individually or in bulk, and disable demo cameras so Live View shows only real feeds — critical for field demos with four physical cameras.

8. Key UI Surfaces

| Surface          | Purpose                                                                   |
|------------------|---------------------------------------------------------------------------|
| Dashboard        | Command center: health, storage days remaining, weekly detections, alerts |
| Live View        | Multi-grid monitoring with LIVE badges and AI overlays                    |
| Playback         | Investigation: zones, timeline, seek, heatmap, clip export                |
| Cameras          | Connect/disconnect devices; demo ON/OFF master switch                     |
| Faces & Plates   | Enrollment + plate search across recordings                               |
| AI Search / Chat | Operator queries in plain English over event history                      |
| Incidents        | Workflow status for critical security events                              |
| Camera Map       | Floor-plan placement; click opens camera evidence                         |

9. Suggested Demonstration Script

1) Login as Admin → Dashboard health cards. 2) Live View multi-grid. 3) Playback → draw a zone → trigger/person event → click timeline marker → auto-seek. 4) AI Search: “people today”. 5) Toggle Demo OFF; note Live View filters to live sources. 6) Export PDF report. 7) Show incident status change. *(Replace this narrative with your recorded walkthrough; keep the video under a clear, well-lit narration.)*

10. Repository & Reproducibility

Source code is organized as a monorepo: backend/ (FastAPI), frontend/ (React), docs/ (architecture, schema, webcam guide), sample-data/, and Docker Compose. API documentation is available at /docs (OpenAPI). Demo accounts: admin / operator / viewer with documented passwords in README. Third-party libraries (Ultralytics YOLO, OpenCV, FastAPI, React, Recharts, ReportLab, EasyOCR) are acknowledged in project documentation.

11. Outcomes & Evaluation Fit

AetherShield fully satisfies the case-study mandate while presenting an operator experience closer to commercial VMS software: unified live+playback, AI events on a navigable timeline, zone-constrained alerting, and a real dashboard. Differentiating modules (NL search, identity, incidents, remote camera ops) strengthen placement-portfolio value without compromising the core evaluation criteria. The architecture is intentionally modular so judges can inspect streaming, AI, and API layers independently.

12. Acknowledgements

Ultralytics YOLOv11; OpenCV; FastAPI / Starlette; React / Vite / Tailwind CSS; Recharts; ReportLab; ByteTrack (via Ultralytics track mode); EasyOCR (optional plate path). Case study brief: A-1 Launchpad 2026 — Smart Video Management System.

**GitHub Repository:** [ INSERT REPOSITORY URL ] | **Video:** [ INSERT VIDEO URL — also on page 1 ]

Prepared for Unstop submission · Maximum 3 content pages (this document) · Cover page & resumes attached separately by the team